Part A §4 — Apple Neural Engine Fallback Strategy (macOS)

> ***Scenario**: On macOS 14+, the Apple Neural Engine (ANE) is a shared system resource. Core ML returns `kCMErrorUnsupportedOperation` during inference — the ANE is unavailable (claimed by another process, thermally throttled, or the model op isn't ANE-compatible).*

Architecture Context

This document covers in-process ANE failure handling within an inference worker (`edge-worker-N`) on macOS Apple Silicon. The worker is one process in the Edge Agent's supervisor-tree model (defined in Part A §1):

- **`edge-supervisor`** — receives tier change notifications from workers via IPC (Unix Domain Sockets, MessagePack). Does not intervene in device fallback — the worker handles escalation autonomously.
- **`edge-gateway`** — routes requests to workers. Reads `device` and `tier` fields from worker heartbeats to adjust queue depth and throughput expectations.
- **`edge-worker-N`** — contains an in-process Device Manager that owns the 4-tier escalation (ANE → Metal GPU → CPU → Cloud). Each tier's device state is tracked independently. The worker process stays alive throughout all escalations.

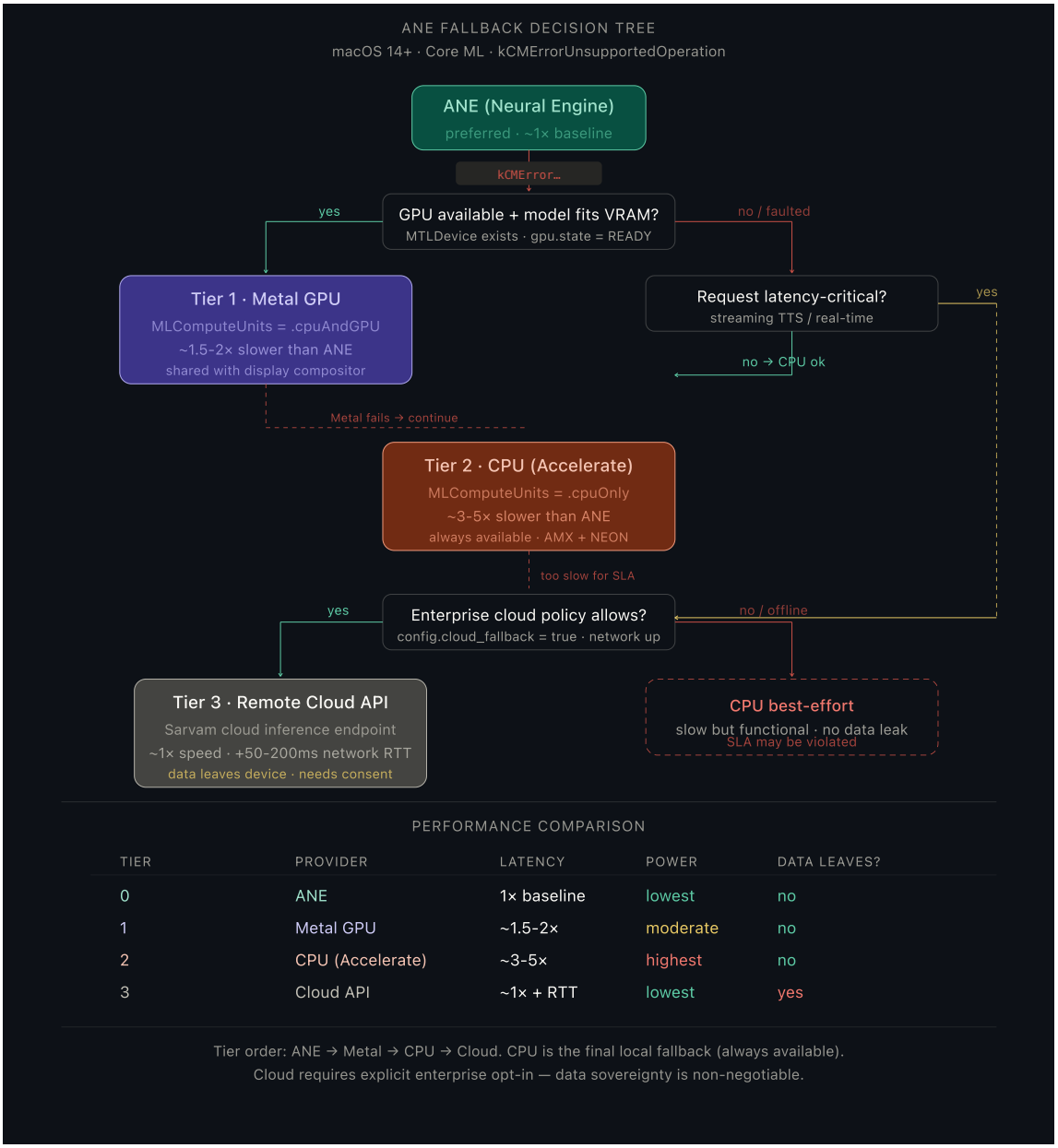
Assumptions

1. **Core ML supports runtime compute unit selection.** Setting `MLModelConfiguration.computeUnits` to `.cpuAndGPU` or `.cpuOnly` forces Core ML to skip the ANE without restarting the process.
2. **ANE unavailability is often transient.** Another process may release the ANE, thermals may recover, or the user may close a competing app. The probe-and-restore cycle is designed for this.
3. **Metal GPU is available on all Apple Silicon Macs.** The M-series SoC includes an integrated GPU accessible via Metal Performance Shaders. It is the primary fallback before CPU.
4. **Cloud fallback is opt-in and requires explicit enterprise policy.** The cloud tier is disabled by default. When enabled, inference data leaves the device — this requires user/enterprise consent and a valid short-lived token from the enterprise provisioning path.
5. **Each device tier's quarantine state is independent.** ANE faulted does not affect GPU or CPU. All 4 tier states are tracked in separate state machines within the same worker process.
6. **Quarantine state is intentionally in-memory only.** If the worker restarts, all tiers reset to READY. Re-detection costs one failed request per faulted tier — acceptable given the transient nature of ANE issues.

Overview

Unlike the Qualcomm NPU scenario (where the device is broken), ANE unavailability on macOS is often **transient** — another process released the ANE, thermals recovered, or the user closed a competing app. The fallback strategy therefore uses a **tiered escalation** with three local tiers and one optional remote tier, each independently health-tracked.

The API contract never changes. The same endpoints, same request format, same response schema. Degradation is communicated entirely through response headers.



Recovery Timeline — ANE → Metal GPU Fallback

Time	Event	Component	Client impact
`t=0ms`	Core ML returns `kCMErrorUnsupportedOperation`	Worker (Core ML)	Request in progress
`t=1ms`	Device Manager marks ANE as `FAULTED`, selects Tier 1 (Metal GPU)	Worker (Device Manager)	None yet
`t=2–10ms`	Core ML model reloaded with `.computeUnits = .cpuAndGPU` (forces Metal)	Worker	None — model file already in memory

`t=10ms`	Same request retried on Metal GPU	Worker	None — client is still waiting
`t=50–100ms`	Metal GPU inference completes (~1.8× slower than ANE)	Worker	Response arrives with `X-Edge-Compute: metal-gpu`
`t=100ms`	IPC notification: `device_event { tier_change: 0→1, ane→metal-gpu }`	Worker → Supervisor	None
`t=100ms+`	Worker heartbeat: `device: metal-gpu`, `tier: 1`	Worker → Gateway	Throughput estimate adjusted (~1.8× slower)
`t=30s`	First health probe: attempt ANE-only inference with test tensor	Worker (Device Manager)	None (background probe)
`t=30s+`	Probe succeeds → restore ANE (Tier 0). Probe fails → backoff to 60s, 120s, ... cap 300s	Worker	If restored: subsequent requests at ANE speed

Recovery Timeline — Full Escalation (ANE → GPU → CPU → Cloud)

Time	Trigger	Tier	Provider	Latency factor
`t=0ms`	ANE: `kCMErrorUnsupportedOperation`	0 → 1	Metal GPU	1.8×
`t=50ms`	GPU: Metal shader compilation fails (rare)	1 → 2	CPU (Accelerate)	4.4×
`t=250ms`	CPU succeeds (CPU never quarantined)	2 (stable)	CPU	4.4×
`t=30s+`	Probe: GPU restored	2 → 1	Metal GPU	1.8×
`t=120s+`	Probe: ANE restored	1 → 0	ANE	1.0×

If cloud is enabled and CPU fails (theoretically impossible but guarded), the system escalates to Tier 3 (cloud inference via `https://api.sarvam.ai/v1/inference`). Cloud latency depends on network RTT.

---

1. Selection Logic — The Fallback Tier List

When ANE fails, the worker runs a synchronous decision function on every request to pick the best available execution provider:

Priority	Tier	Provider	Core ML `MLComputeUnits`	When chosen
0	**ANE**	Neural Engine	`.all`	Default — Core ML picks ANE when available

1	**Metal GPU**	Apple GPU via Metal	`.cpuAndGPU`	ANE faulted, GPU device present and READY
2	**CPU**	Accelerate (AMX + NEON)	`.cpuOnly`	ANE + GPU both faulted, or non-latency-critical request
3	**Cloud**	Sarvam cloud API	N/A (HTTP)	Latency-critical + local too slow + enterprise policy allows

The decision function

```
func selectProvider(for request: InferenceRequest) -> ComputeProvider {
    // Tier 0: ANE (via Core ML .all — lets Core ML use ANE if available)
    if deviceManager.state(of: .ane) == .ready {
        return .all
    }

    // Tier 1: Metal GPU
    if deviceManager.state(of: .gpu) == .ready,
       modelFitsInUnifiedMemory(request.modelId) {
        return .cpuAndGPU
    }

    // Tier 2 vs Tier 3 decision
    if request.isLatencyCritical,
       config.cloudFallbackAllowed,
       networkMonitor.isReachable {
        return .cloud
    }

    // Tier 2: CPU (always available, always works)
    return .cpuOnly
}
```

Why this order?

Metal GPU before CPU: On Apple Silicon, the GPU shares unified memory with the CPU and the ANE. Metal compute shaders can run neural network ops ~2× faster than pure CPU because the GPU has more ALUs and higher memory bandwidth to the same physical RAM. The tradeoff: the GPU is shared with the display compositor and any other Metal workloads (video editing, etc.), so it may be partially occupied.

CPU before Cloud: Data sovereignty. Edge's entire value proposition is on-device inference — data never leaves the machine. Cloud is the escape hatch for latency-critical requests when local compute is too slow, but it requires explicit enterprise opt-in (`config.cloud_fallback = true`). Many enterprises will disable this entirely.

Cloud is never the default: Even with the flag enabled, cloud is only chosen for `latency_critical` requests (streaming TTS, real-time translation). Batch jobs always run locally, even if slow.

---

2. Performance Tradeoffs

Latency per tier

Tier	Token generation (TTS)	Batch translate (NMT)	Power draw	Thermal impact
ANE	~8ms/token	~40ms/batch	Very low	Negligible
Metal GPU	~14ms/token	~70ms/batch	Moderate	Fans may spin up
CPU (Accelerate)	~35ms/token	~180ms/batch	High	Sustained thermals
Cloud API	~10ms/token + RTT	~50ms + RTT	Very low (local)	None

Key tradeoff: Metal GPU vs CPU

Factor	Metal GPU	CPU
Speed	~1.5-2× slower than ANE	~3-5× slower than ANE
Availability	May be contested (video, UI compositing)	Always free
Power	Moderate — GPU frequency scales	High — all P-cores active
Model compatibility	Some ops not Metal-compatible	All ops supported
Side effects	Can cause UI jank if GPU is saturated	Can cause system slowness

The worker monitors GPU utilization via `MTLDevice.currentAllocatedSize` and the IOKit GPU residency counters. If the GPU is >80% utilized by other workloads, the fallback selector skips straight to CPU to avoid causing UI jank.

Key tradeoff: CPU vs Cloud

Factor	CPU	Cloud
Latency	Predictable (3-5×)	Variable (depends on network)
Data	Stays on device	Leaves device
Availability	Always	Requires network + auth + quota
Cost	Free	Per-request billing
Compliance	Always compliant	Requires enterprise consent

---

3. Failure Escalation Order

Each tier can independently fail. The escalation is a waterfall — if a tier fails, the worker drops to the next tier within the **same request lifecycle**. No inter-process communication, no supervisor involvement.

```
Request arrives
  │
  ├─ Try Tier 0 (ANE via .all)
  │    └─ kCMErrorUnsupportedOperation → quarantine ANE
  │
  ├─ Try Tier 1 (Metal GPU via .cpuAndGPU)
  │    └─ MTLCommandBuffer error / GPU timeout → quarantine GPU
  │
  ├─ Try Tier 2 (CPU via .cpuOnly)
  │    └─ Always succeeds (Accelerate framework is a software library)
  │    └─ If latency_critical and exceeds SLA → consider Tier 3
  │
  └─ Try Tier 3 (Cloud API)
       └─ Network timeout / 5xx → return CPU result anyway (best effort)
```

Escalation within a single request

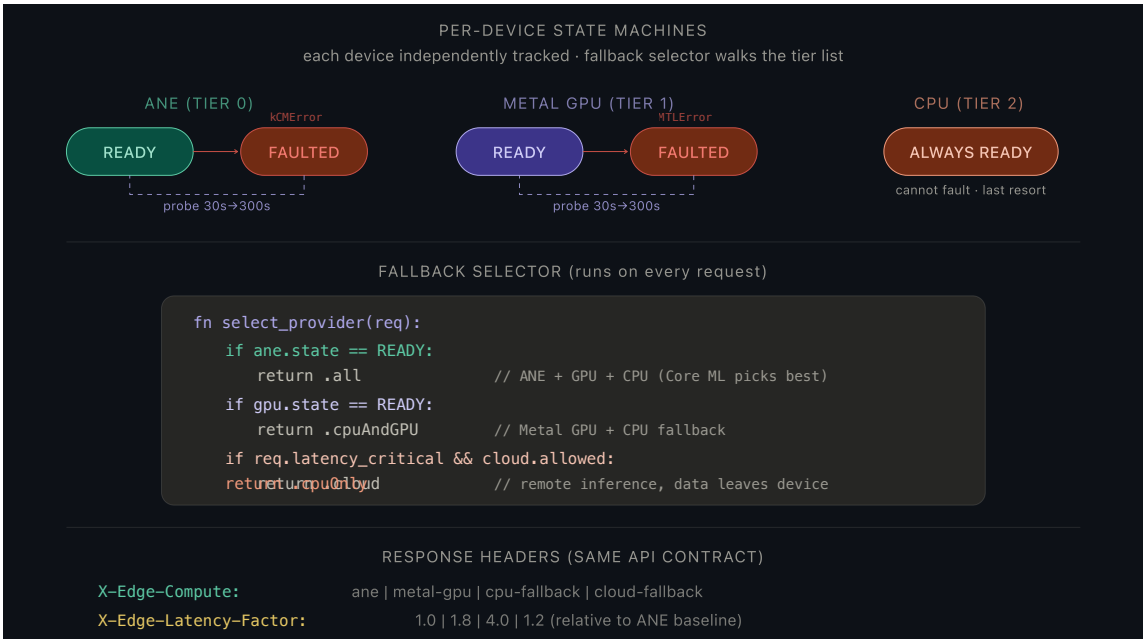
The critical design choice: **a single request can cascade through multiple tiers**. The worker catches the error from each tier and immediately retries on the next, without returning to the gateway.

```swift
func runInference(_ request: InferenceRequest) throws -> InferenceResult {
    let tiers: [ComputeProvider] = buildTierList(for: request)

    for tier in tiers {
        do {
            let result = try executeOnProvider(request, provider: tier)
            result.metadata.computeProvider = tier
            return result
        } catch let error as CoreMLError where error.isDeviceUnavailable {
            deviceManager.quarantine(tier.deviceType)
            continue  // try next tier
        } catch let error as MTLError {
            deviceManager.quarantine(.gpu)
            continue
        }
        // CPU tier doesn't throw device errors — it always works
    }

    // Unreachable: CPU tier always succeeds
    fatalError("All tiers exhausted — this shouldn't happen")
}
```

The worst case: ANE fails → GPU fails → CPU succeeds. Total overhead for the double-fallback: ~50ms of session reinitialization. The client sees ~50ms extra latency on that one request, then all subsequent requests go directly to the highest available tier (no more fallback overhead).

Per-device quarantine

Each device has its own state machine, identical to the NPU quarantine from Part A §3:



Device	Quarantine behavior	Probe interval	Probe method
ANE	Quarantine on `kCMError`. Probe with `.all` compute units on a canary model.	30s → 300s (exp backoff)	Canary Core ML inference with `.all`
Metal GPU	Quarantine on `MTLCommandBuffer` error or GPU timeout.	30s → 300s (exp backoff)	Small Metal compute shader + result check
CPU	Never quarantined — always available	N/A	N/A
Cloud	Quarantine on network timeout or 5xx.	15s → 60s (shorter backoff)	`HEAD` request to health endpoint

ANE quarantine probes are shorter than the NPU case because ANE unavailability is often transient (another process released the ANE). The probe checks if Core ML will schedule ops on the ANE by running a canary model with `.all` and inspecting the `MLPredictionOptions` output to confirm ANE was actually used.

---

4. Communicating Degraded Mode Without Changing the API Contract

Principle: same shape, different metadata

The response body schema is **identical** regardless of which tier served the request. A TTS response always returns audio bytes. A translation response always returns text. The API contract (endpoint paths, request format, response schema, error codes) never changes.

Degradation is communicated exclusively through **response headers** and an **optional metadata field** in the JSON body:

Response headers (always present)

```
HTTP/1.1 200 OK
Content-Type: application/json
X-Edge-Compute: metal-gpu
X-Edge-Latency-Factor: 1.8
X-Edge-Device-Status: degraded
X-Edge-Estimated-Restore: 28
X-Edge-Tier: 1
```

Header	Values	Purpose
`X-Edge-Compute`	`ane` | `metal-gpu` | `cpu-fallback` | `cloud-fallback`	Which provider served this request
`X-Edge-Latency-Factor`	Float	Slowdown relative to ANE baseline
`X-Edge-Device-Status`	`healthy` | `degraded` | `critical`	`degraded` = ANE down, GPU serving. `critical` = CPU-only.
`X-Edge-Estimated-Restore`	Integer (seconds)	Time until the next ANE health probe
`X-Edge-Tier`	`0` | `1` | `2` | `3`	Numeric tier for programmatic consumption

Optional response body metadata

For clients that want richer info without parsing headers, the response body includes an optional `_meta` field:

```
{
  "text": "नमस्ते, मैं आपकी मदद कर सकता हूँ।",
  "language": "hi",
  "_meta": {
    "compute": "metal-gpu",
    "tier": 1,
    "latency_ms": 142,
    "baseline_latency_ms": 80,
    "device_status": "degraded",
    "degraded_since": "2025-01-15T10:23:45Z"
  }
}
```

The `_meta` field is **always present** (even when healthy — with `tier: 0`). Clients that don't care about it ignore it. Clients that do (e.g., an enterprise dashboard showing device health) read it.

WebSocket / SSE streaming

For token streams, a single metadata frame at the start:

```
{"type": "meta", "compute": "cpu-fallback", "tier": 2, "latency_factor": 4.0,
"device_status": "critical"}
{"type": "token", "text": "न"}
{"type": "token", "text": "म"}
{"type": "token", "text": "स्ते"}
{"type": "done", "total_ms": 850}
```

Health endpoint (unchanged)

```
GET /v1/health/devices
```

```
{
  "ane": { "state": "faulted", "since": "2025-01-15T10:23:45Z", "next_probe_s": 28
},
  "gpu": { "state": "ready" },
  "cpu": { "state": "ready" },
  "cloud": { "state": "ready", "policy": "allowed", "reachable": true },
  "active_tier": 1,
  "active_provider": "metal-gpu"
}
```

What the enterprise client should do

The client is **not required** to do anything — the response is always `200 OK` with valid data. But well-behaved clients can:

1. Read `X-Edge-Device-Status` and show a subtle "reduced performance" indicator
2. Read `X-Edge-Latency-Factor` and adjust UI timeouts accordingly
3. Read `X-Edge-Estimated-Restore` and schedule a retry at normal speed
4. Poll `/v1/health/devices` to monitor the fleet of Edge installations

None of this changes the API contract. A client written before the fallback system existed will continue to work — it just won't know about the degradation.

---

5. Scheduler Interaction During ANE Fallback

Like the NPU fallback case, the worker stays alive during ANE fallback. The scheduler adapts based on heartbeat data:

1. **Worker stays in pool** — no removal, no re-registration. The worker reports its current compute tier in each heartbeat.
2. **Throughput estimate adjusts** — the scheduler reads the `device` field from heartbeats. When it changes from `ane` to `metal-gpu` or `cpu`, the scheduler recalculates expected request duration:
   - `ane` → baseline
   - `metal-gpu` → 1.8× baseline
   - `cpu` → 4× baseline
   - `cloud` → 1× + variable RTT
3. **Queue depth scales** — if the effective drain rate drops below a threshold, the scheduler reduces the backpressure limit proportionally (e.g., 64 → 32 at Tier 2, 64 → 16 at CPU-only with both ANE

and GPU faulted).

4. **No re-routing** — the same worker handles the request. Sticky routing continues normally since the model is still loaded in the same worker's memory.
5. **Tier restoration is transparent** — when a health probe succeeds and the worker restores a higher tier, the next heartbeat reflects the change. The scheduler restores the normal queue depth automatically.

---

6. State/Event Payload Examples

Device Manager internal state (macOS — all 4 tiers)

```
{
  "ane": {
    "state": "FAULTED",
    "faulted_at": "2025-01-15T10:23:45Z",
    "error": "kCMErrorUnsupportedOperation",
    "probe_attempt": 1,
    "next_probe_at": "2025-01-15T10:24:45Z",
    "backoff_interval_s": 60
  },
  "gpu": {
    "state": "READY",
    "utilization_pct": 42,
    "metal_device": "Apple M3 Pro"
  },
  "cpu": {
    "state": "READY",
    "cores": 12,
    "performance_cores": 6
  },
  "cloud": {
    "state": "READY",
    "policy": "allowed",
    "endpoint": "https://api.sarvam.ai/v1/inference",
    "reachable": true
  },
  "active_tier": 1,
  "active_provider": "metal-gpu"
}
```

Worker → Supervisor (tier change event via IPC)

```
{
  "type": "device_event",
  "worker_id": "worker-0",
  "event": "tier_change",
  "from_tier": 0,
  "to_tier": 1,
  "from_provider": "ane",
```

```
  "to_provider": "metal-gpu",
  "reason": "kCMErrorUnsupportedOperation",
  "timestamp": "2025-01-15T10:23:45.123Z"
}
```

Worker → Supervisor (ANE restored event)

```
{
  "type": "device_event",
  "worker_id": "worker-0",
  "event": "tier_change",
  "from_tier": 1,
  "to_tier": 0,
  "from_provider": "metal-gpu",
  "to_provider": "ane",
  "reason": "health_probe_passed",
  "probe_attempts": 2,
  "downtime_s": 90,
  "timestamp": "2025-01-15T10:25:15.456Z"
}
```

Worker → Supervisor (heartbeat during degraded state)

```
{
  "type": "heartbeat",
  "worker_id": "worker-0",
  "state": "idle",
  "model_loaded": "indic-tts-v3",
  "uptime_ms": 847200,
  "memory_mb": 980,
  "device": "metal-gpu",
  "tier": 1,
  "devices_faulted": ["ane"]
}
```

Worker → Gateway (cloud fallback inference response)

```
{
  "type": "inference_response",
  "request_id": "req_9b2f",
  "status": "ok",
  "result": "<binary audio data>",
  "compute_provider": "cloud",
  "tier": 3,
  "latency_ms": 95,
  "cloud_rtt_ms": 45,
  "model_id": "indic-tts-v3"
}
```

7. Design Tradeoff Explanations

Decision	Chosen	Alternative	Rationale
4-tier waterfall (ANE → GPU → CPU → Cloud)	Ordered tier list with per-request cascade	Binary fallback (ANE or CPU only)	Metal GPU is 2× faster than CPU on Apple Silicon. Skipping it wastes available performance. Cloud covers the latency-critical edge case.
In-process cascade (no restart)	Catch error, retry next tier in same function	Kill worker, restart with different compute flag	Input tensors already in memory. ~50ms cascade vs ~3s restart. Same model, same process.
Per-device independent quarantine	Each tier has its own state machine	Single "degraded" flag for the whole worker	ANE can recover while GPU is faulted (different failure modes). Independent tracking avoids unnecessary downgrades.
GPU utilization threshold (80%)	Skip GPU if >80% utilized	Always try GPU if READY	Saturating the GPU causes UI jank (compositing, animation). Skipping to CPU avoids visible user impact.
Cloud requires explicit opt-in	`config.cloud_fallback = true` + `latency_critical`	Cloud as automatic fallback	Data sovereignty is Edge's core value. Cloud means data leaves the device — must be an intentional enterprise decision.
Shorter cloud backoff (15s → 60s)	Faster probe cycle for network tier	Same 30s → 300s as local devices	Network issues resolve faster than hardware faults. Quick re-probe catches transient connectivity drops.
CPU never quarantined	CPU is always the last resort	Allow CPU quarantine on sustained thermal throttle	Accelerate is a software library — it can't "fail" in the device sense. Thermal throttling slows it but doesn't break it.
Same API contract across all tiers	Degradation via headers + `_meta` only	Different response schemas per tier	Clients written before the fallback system still work unchanged. No version negotiation. No breaking changes.

8. Performance & Reliability

Metric	ANE (Tier 0)	Metal GPU (Tier 1)	CPU (Tier 2)	Cloud (Tier 3)	Notes
Token generation (TTS)	~8ms/token	~14ms/token	~35ms/token	~10ms + RTT	RTT typically 20-80ms
Batch translation (NMT)	~40ms	~70ms	~180ms	~50ms + RTT	
Session swap time	N/A	~30ms	~20ms	N/A (HTTP)	Core ML recompile for new compute units
Detection latency	N/A	~0ms (sync)	~0ms (sync)	~0ms (sync)	Same event loop tick
Cascade overhead (worst case)	N/A	~50ms	~80ms	~120ms	ANE fail → GPU fail → CPU succeed
Health probe overhead	~3ms	~5ms	N/A	~20ms	Canary model or HEAD request
Worst-case single-request	8ms	**64ms**	**115ms**	**130ms + RTT**	Includes cascade + inference

Reliability guarantees

- **No request lost**: every request cascades to at least CPU, which always succeeds
- **No process restart**: all tier changes are in-process session swaps
- **Bounded degradation**: CPU is 4× slower than ANE but deterministic — no variable latency
- **Data sovereignty preserved**: cloud is never used unless explicitly opted in by the enterprise
- **Independent recovery**: each device is probed and restored independently — GPU recovery doesn't require ANE recovery
- **Silent corruption caught**: canary result validation on ANE and GPU probes prevents serving garbage from a post-throttle device
- **Graceful under contention**: GPU utilization check prevents the inference worker from causing UI jank

---

Summary

Question	Answer
Selection logic	Tiered waterfall: ANE → Metal GPU → CPU → Cloud. Per-request decision based on device state, GPU utilization, latency criticality, and enterprise cloud policy.
Performance tradeoffs	GPU: 1.5-2× slower, shares with display. CPU: 3-5× slower, always available. Cloud: ~1× + RTT, data leaves device.

Escalation order	In-process cascade within a single request. ANE fail → try GPU → try CPU → try Cloud. CPU always succeeds. ~50ms overhead for double-fallback.
Client communication	`X-Edge-Compute` + `X-Edge-Latency-Factor` + `X-Edge-Device-Status` headers. `_meta` field in response body. `/v1/health/devices` endpoint. API contract unchanged — always `200 OK`.

---

9. Offline-Only Mode and Cloud Authentication Policy

Cloud fallback is policy-gated and never implicit.

9.1 Policy matrix

Enterprise policy	Local tiers available	Action
`cloud_fallback = false`	Any local tier available	Continue local execution (ANE/GPU/CPU)
`cloud_fallback = false`	No local tier available (unexpected)	Return deterministic local failure code with remediation hint
`cloud_fallback = true`	Local tiers degraded + request is latency-critical	Allow Tier 3 cloud fallback

9.2 Cloud auth contract

1. Shell/runtime obtains short-lived token from enterprise provisioning path.
2. Token is stored in OS keychain only, never in plaintext config.
3. Cloud fallback requests include correlation IDs and minimum payload required for inference.
4. `401/403` auth failures quarantine cloud tier separately and do not block local CPU fallback.

---

10. Thermal and Memory Guardrails (Apple Silicon)

To avoid user-visible system impact during degradation, tier selection includes thermal and unified-memory constraints.

10.1 GPU admission check

GPU tier is considered eligible only when:

1. Unified memory headroom is above configured threshold.
2. Real-time GPU utilization is below contention threshold.
3. Current thermal state is not `serious` or `critical`.

If any check fails, runtime skips GPU tier and proceeds to CPU tier.

10.2 Guardrail payload example

```
{
  "gpu": {
    "state": "READY",
    "utilization_pct": 83,
```

```
    "unified_mem_free_mb": 1024,
    "admission": "denied_high_utilization"
  },
  "thermal": {
    "state": "serious",
    "policy": "prefer_cpu"
  }
}
```

These controls protect UX quality while keeping inference availability high.

Sarvam AI — Edge Runtime Team — Backend Intern Assignment